MOBILE DEVELOPMENT · RIVERPOD MODULE · LAB SHEET

Real-Time Data Ticker with `AsyncValue`

Loading, data, error and refresh states through Riverpod pattern matching

AUTHOR	Mukhammadali Khayotov	TIME	1 h 40 min
SESSIONS	Monday 19 October 2026 · Thursday 22 October 2026		
SCORING	60 pts = 100 %	PASS MARK	20 pts
STUDENT		PARTNER	

Overview

Build a live price ticker that streams simulated market data and renders every `AsyncValue` state (loading, data, error, refreshing) through Dart 3 pattern matching on Riverpod providers.

DURATION	1 hour 40 minutes (5 timed parts + optional stretch)
STACK	Flutter 3.x, Dart 3.x, `flutter_riverpod` 3.x
PREREQUISITES	Completed last week's Riverpod lab; comfortable with `ConsumerWidget`, `ref.watch`, and basic `StreamProvider` usage
DELIVERABLE	A runnable app plus passing tests, pushed to your lab repo before the end of session

By the end you will be able to:

1. Model a tick with a `freezed`-free immutable class and expose it through a `StreamProvider`.
2. Destructure `AsyncValue` with `switch` expressions and object patterns (`AsyncData(:final value)`).
3. Distinguish first load, refresh, and reload using `isLoading`, `isRefreshing`, and `hasValue`.
4. Keep the last good data on screen while an error or refresh is in flight.
5. Write widget and provider tests that override the stream to force each state.

Work in pairs. Commit at the end of each part with the tag shown in that part's header.

1 Setup

10 MIN, TAG P1-SETUP

Create a fresh project and confirm a `ProviderScope` boots with no errors before writing any ticker code.

1. Create the app and add dependencies:

```
flutter create riverpod_ticker --empty
cd riverpod_ticker
flutter pub add flutter_riverpod
flutter pub add --dev flutter_test riverpod_test mocktail
```

2. Replace `lib/main.dart`:

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'ticker_page.dart';

void main() => runApp(const ProviderScope(child: TickerApp()));

class TickerApp extends StatelessWidget {
  const TickerApp({super.key});

  @override
  Widget build(BuildContext context) => MaterialApp(
        title: 'Ticker',
        theme: ThemeData(colorSchemeSeed: Colors.teal, useMaterial3: true),
        home: const TickerPage(),
      );
}
```

3. Create the folder layout you will fill in later:

```
lib/
  main.dart
  models/tick.dart
  services/tick_feed.dart
  providers/ticker_providers.dart
  ticker_page.dart
  widgets/tick_row.dart
test/
  ticker_providers_test.dart
  ticker_page_test.dart
```

4. Stub `ticker_page.dart` with a `ConsumerWidget` that shows a `Scaffold` and the text "Ticker". Run `flutter run` and confirm the page renders.

> CHECKPOINT `flutter analyze` reports no issues. Commit.

2 Model and stream provider

20 MIN, TAG P2-PROVIDER

The feed is a simulated service that emits a new `Tick` every 500 ms and throws on demand, so you can reproduce every state without a network.

2.1 The model (`lib/models/tick.dart`)

```dart
class Tick {
  const Tick({required this.symbol, required this.price, required this.change, required this.at});

  final String symbol;
  final double price;
  final double change; // delta vs previous tick
  final DateTime at;

  bool get isUp => change >= 0;
}
```

2.2 The feed (`lib/services/tick_feed.dart`)

```dart
import 'dart:async';
import 'dart:math';
import '../models/tick.dart';

class FeedException implements Exception {
  const FeedException(this.message);
  final String message;
  @override
  String toString() => message;
}

class TickFeed {
  TickFeed({this.symbols = const ['BTC', 'ETH', 'SOL'], this.failAfter});

  final List<String> symbols;
  final int? failAfter; // emit this many ticks, then throw
  final _rng = Random();
  final _prices = <String, double>{};

  Stream<List<Tick>> watch() async* {
    var count = 0;
    await Future<void>.delayed(const Duration(milliseconds: 800)); // visible first load
    while (true) {
      if (failAfter != null && count >= failAfter!) {
        throw const FeedException('Feed disconnected');
      }
      yield [for (final s in symbols) _next(s)];
      count++;
      await Future<void>.delayed(const Duration(milliseconds: 500));
    }
  }

  Tick _next(String symbol) {
    final prev = _prices[symbol] ?? 100 + _rng.nextDouble() * 900;
    final change = (_rng.nextDouble() - 0.5) * 4;
    final price = (prev + change).clamp(1, double.infinity).toDouble();
    _prices[symbol] = price;
    return Tick(symbol: symbol, price: price, change: change, at: DateTime.now());
  }
}
```

2.3 Providers (`lib/providers/ticker_providers.dart`)

```dart
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../models/tick.dart';
import '../services/tick_feed.dart';

/// Overridable in tests and in the error demo.
final tickFeedProvider = Provider<TickFeed>((ref) => TickFeed());

final tickerProvider = StreamProvider.autoDispose<List<Tick>>((ref) {
  return ref.watch(tickFeedProvider).watch();
});
```

QUESTIONS TO ANSWER IN YOUR LAB NOTES BEFORE MOVING ON

- Why is the feed behind its own `Provider` instead of being constructed inside `tickerProvider`?
- What does `autoDispose` do when the user navigates away mid-stream?
- What `AsyncValue` does `tickerProvider` hold during the 800 ms delay, and what is `hasValue` at that moment?

CHECKPOINT a temporary `ref.listen(tickerProvider, (_, next) => debugPrint('$next'))` in `TickerPage` prints `AsyncLoading`, then a stream of `AsyncData`. Remove it, then commit.

3 Ticker UI with `AsyncValue` pattern matching

30 MIN, TAG P3-UI

`AsyncValue` is a sealed class with three subtypes (`AsyncData`, `AsyncLoading`, `AsyncError`), so a `switch` expression is exhaustive and the compiler tells you when you forget a state.

3.1 The row widget (`lib/widgets/tick_row.dart`)

```dart
import 'package:flutter/material.dart';
import '../models/tick.dart';

class TickRow extends StatelessWidget {
  const TickRow(this.tick, {super.key});
  final Tick tick;

  @override
  Widget build(BuildContext context) {
    final color = tick.isUp ? Colors.green : Colors.red;
    final arrow = tick.isUp ? '▲' : '▼';
    return ListTile(
      leading: CircleAvatar(child: Text(tick.symbol.substring(0, 1))),
      title: Text(tick.symbol),
      subtitle: Text(tick.price.toStringAsFixed(2)),
      trailing: Text(
        '$arrow ${tick.change.abs().toStringAsFixed(2)}',
        style: TextStyle(color: color, fontWeight: FontWeight.bold),
      ),
    );
  }
}
```

3.2 First pass: the three-way switch (`lib/ticker_page.dart`)

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'providers/ticker_providers.dart';
import 'widgets/tick_row.dart';

class TickerPage extends ConsumerWidget {
  const TickerPage({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final ticks = ref.watch(tickerProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('Live ticker')),
      body: switch (ticks) {
        AsyncData(:final value) => ListView(
            children: [for (final t in value) TickRow(t, key: ValueKey(t.symbol))],
          ),
        AsyncError(:final error) => Center(child: Text('Error: $error')),
        AsyncLoading() => const Center(child: CircularProgressIndicator()),
      },
    );
  }
}
```

Run it. You should see a spinner for roughly a second, then rows updating twice a second.

3.3 Exercise: break the switch on purpose

Comment out the `AsyncLoading()` arm and run `flutter analyze`. Record the exact diagnostic in your notes — this is the compile-time guarantee you are relying on.

3.4 Exercise: pattern-match on content, not just state

Add two arms *above* `AsyncData(:final value)` so that an empty list and a single-symbol list render differently:

```
AsyncData(value: []) => const Center(child: Text('No symbols subscribed')),
AsyncData(value: [final only]) => Center(child: TickRow(only)),
```

Verify the empty arm by temporarily overriding the feed with `TickFeed(symbols: const [])` in `ProviderScope.overrides`. Why must these arms come before the general `AsyncData` arm?

> CHECKPOINT all three states are reachable (use `TickFeed(failAfter: 5)` to see the error). Commit.

4 Errors, retry, and preserving previous data

25 MIN, TAG P4-RESILIENCE

A real ticker never blanks the screen when the feed hiccups; it keeps the last prices visible, flags the problem, and offers a retry. `AsyncValue` carries the previous value through `AsyncError` and `AsyncLoading`, so you can do this without extra state.

4.1 Make the error reachable in the app

Add a `failAfter` toggle so demos and tests don't require code edits:

```
// ticker_providers.dart
final failAfterProvider = StateProvider<int?>((ref) => null);

final tickFeedProvider = Provider<TickFeed>((ref) {
  return TickFeed(failAfter: ref.watch(failAfterProvider));
});
```

In the `AppBar`, add an `IconButton` that sets `failAfterProvider` to `5`, and a second that sets it back to `null`.

4.2 Rewrite the body with guards and `isRefreshing`

Replace the `switch` from Part 3 with one that keys on both the state and whether old data exists:

```
body: switch (ticks) {
  // Error with stale data: show rows dimmed, banner on top
  AsyncError(:final error, :final value?) => Column(children: [
      MaterialBanner(
        content: Text('Feed error: $error — showing last prices'),
        actions: [
          TextButton(
            onPressed: () => ref.invalidate(tickerProvider),
            child: const Text('Retry'),
          ),
        ],
      ),
      Expanded(
        child: Opacity(
          opacity: 0.5,
          child: ListView(children: [for (final t in value) TickRow(t, key: ValueKey(t.symbol))]),
        ),
      ),
    ]),

  // Error with nothing to show
  AsyncError(:final error) => Center(
      child: Column(mainAxisSize: MainAxisSize.min, children: [
        Text('Could not load ticker\n$error', textAlign: TextAlign.center),
        const SizedBox(height: 12),
        FilledButton(onPressed: () => ref.invalidate(tickerProvider), child: const Text('Retry')),
      ]),
    ),

  // Refresh in progress, keep rows, show thin progress bar
  AsyncValue(:final value?, isLoading: true) => Column(children: [
      const LinearProgressIndicator(),
      Expanded(child: ListView(children: [for (final t in value) TickRow(t, key: ValueKey(t.symbol))])),
    ]),

  AsyncData(:final value) => ListView(children: [for (final t in value) TickRow(t, key: ValueKey(t.symbol))]),

  AsyncLoading() => const Center(child: CircularProgressIndicator()),
},
```

NOTES ON THE PATTERNS

- `:final value?` is a null-check pattern: the arm matches only when `value` is non-null, i.e. a previous emission exists.
- `AsyncValue(:final value?, isLoading: true)` matches any subtype that is loading *and* has old data. This covers `ref.refresh` and `ref.invalidate`.
- Arm order is a priority list; the specific arms must sit above the general ones.

4.3 Exercise: `refresh` vs `invalidate`

Wrap the list in a `RefreshIndicator` whose `onRefresh` is `() => ref.refresh(tickerProvider.future)`. Then swap it for `ref.invalidate(tickerProvider)`. Record in your notes which one keeps the previous rows visible during reload, and why `refresh(...future)` returns something awaitable.

4.4 Exercise: distinguish a reconnect from a first load

After a retry, the spinner should not take over the full screen if prices were ever shown. Confirm your arms already handle this; if not, fix the order. Then explain in one sentence the difference between `isRefreshing` and `isReloading` on `AsyncValue`.

> CHECKPOINT with `failAfter = 5` the banner appears over dimmed rows, Retry restores live data, and the full-screen spinner only ever shows on cold start. Commit.

5 Tests

15 MIN, TAG P5-TESTS

Override `tickerProvider` with hand-built streams so each `AsyncValue` state is forced deterministically; no timers, no randomness.

5.1 Provider test (`test/ticker_providers_test.dart`)

```dart
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:riverpod_ticker/providers/ticker_providers.dart';
import 'package:riverpod_ticker/services/tick_feed.dart';

void main() {
  test('starts loading, then emits data', () async {
    final container = ProviderContainer(overrides: [
      tickFeedProvider.overrideWithValue(TickFeed(symbols: const ['BTC'])),
    ]);
    addTearDown(container.dispose);

    expect(container.read(tickerProvider), isA<AsyncLoading<Object?>>());

    final first = await container.read(tickerProvider.future);
    expect(first.single.symbol, 'BTC');
    expect(container.read(tickerProvider).hasValue, isTrue);
  });

  test('surfaces FeedException as AsyncError with previous value', () async {
    final container = ProviderContainer(overrides: [
      tickFeedProvider.overrideWithValue(TickFeed(symbols: const ['BTC'], failAfter: 1)),
    ]);
    addTearDown(container.dispose);

    final sub = container.listen(tickerProvider, (_, __) {});
    await container.read(tickerProvider.future);
    await Future<void>.delayed(const Duration(milliseconds: 600));

    final state = container.read(tickerProvider);
    expect(state, isA<AsyncError<Object?>>());
    expect(state.hasValue, isTrue, reason: 'previous ticks are kept');
    expect(state.error, isA<FeedException>());
    sub.close();
  });
}
```

5.2 Widget tests (`test/ticker_page_test.dart`)

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:riverpod_ticker/models/tick.dart';
import 'package:riverpod_ticker/providers/ticker_providers.dart';
import 'package:riverpod_ticker/ticker_page.dart';

Widget harness(Override o) => ProviderScope(
      overrides: [o],
      child: const MaterialApp(home: TickerPage()),
    );

final sample = [Tick(symbol: 'BTC', price: 100, change: 1.5, at: DateTime(2026))];

void main() {
  testWidgets('loading shows spinner', (tester) async {
    await tester.pumpWidget(harness(
      tickerProvider.overrideWith((ref) => const Stream<List<Tick>>.empty()),
    ));
    expect(find.byType(CircularProgressIndicator), findsOneWidget);
  });

  testWidgets('data shows rows', (tester) async {
    await tester.pumpWidget(harness(
      tickerProvider.overrideWith((ref) => Stream.value(sample)),
    ));
    await tester.pump();
    expect(find.text('BTC'), findsOneWidget);
    expect(find.textContaining('▲'), findsOneWidget);
  });

  testWidgets('error without data shows retry', (tester) async {
    await tester.pumpWidget(harness(
      tickerProvider.overrideWith((ref) => Stream<List<Tick>>.error('boom')),
    ));
    await tester.pump();
    expect(find.text('Retry'), findsOneWidget);
    expect(find.byType(MaterialBanner), findsNothing);
  });
}
```

5.3 Exercise: the hard case

Write a fourth widget test, `error after data shows banner over rows`. Use a `StreamController<List<Tick>>`, add `sample`, pump, then `addError`, pump, and assert both `MaterialBanner` and `BTC` are present. This is the test that proves Part 4 works.

> CHECKPOINT `flutter test` passes 6 tests. Commit and push.

Stretch goals (if you finish early)

1. **Per-symbol providers.** Add `tickProvider = StreamProvider.autoDispose.family<Tick, String>` built with `ref.watch(tickerProvider.stream)` filtered by symbol, and make each `TickRow` a `ConsumerWidget` that watches only its own symbol. Observe which rows rebuild using the Riverpod DevTools.
2. **Flash on change.** Animate the row background green or red for 300 ms when the price moves, using `ref.listen` inside the row rather than `setState`.
3. **Backoff retry.** Replace the manual Retry button with an exponential backoff inside the provider (`1s, 2s, 4s`, max 3 attempts) and expose the attempt count in the banner text.
4. **`AsyncValue.guard`.** Add a one-shot `FutureProvider` that fetches "account balance" (simulated) and wrap it in `AsyncValue.guard`; compare how the error arrives versus the stream case.

Checkpoint questions (answer in `NOTES.md`)

1. Which three subtypes make `AsyncValue` sealed, and what does the analyzer report when a `switch` expression misses one?
2. When does `AsyncError.value` carry a non-null value, and when is it null?
3. What is the difference between `ref.refresh(provider)`, `ref.refresh(provider.future)` and `ref.invalidate(provider)`?
4. Why does `AsyncValue(:final value?, isLoading: true)` need to sit above `AsyncData(:final value)`?
5. Give one reason to prefer `switch` expressions over `.when(...)` for `AsyncValue`, and one situation where `.when` is still convenient.

Submission checklist

- ☐ Tags `p1-setup` through `p5-tests` present in the repo
- ☐ `flutter analyze` clean
- ☐ `flutter test` passes at least 6 tests, including the error-after-data widget test
- ☐ `NOTES.md` answers the five checkpoint questions plus the inline questions from Parts 2, 3 and 4

- [ ] Screen recording (under 30 s) showing cold load, live ticks, forced error with banner, and a successful retry

Grading

The lab is scored in points; 60 points earned within the 1 h 40 min session equals a full 100 %, and 20 points is the minimum to pass.

ITEM	POINTS	AWARDED WHEN
Part 1 — Setup	5	Tag `p1-setup` pushed, `flutter analyze` clean
Part 2 — Model and provider	15	Tag `p2-provider` pushed, checkpoint log observed, three inline questions answered
Part 3 — Ticker UI	20	Tag `p3-ui` pushed, all three states reachable, exercises 3.3 and 3.4 done
Part 4 — Resilience	20	Tag `p4-resilience` pushed, banner over dimmed rows, retry works, exercises 4.3 and 4.4 done
Part 5 — Tests	10	Tag `p5-tests` pushed, 6 tests pass including error-after-data
`NOTES.md`	5	All five checkpoint questions answered
Stretch goal (each)	5	Working and demonstrated; up to 4 goals = 20
Screen recording	5	Under 30 s, shows cold load, ticks, error, retry

Maximum available: 100 points.

RULES

1. Points count only for work committed, pushed and demonstrated to the instructor before the session ends.
2. Score = min(points, 60) ÷ 60 × 100 %. Reaching 60 points gives 100 %; points above 60 do not raise the score.
3. Fewer than 20 points is a fail; the lab must be redone and resubmitted at the next session.
4. Partial credit within an item is at the instructor's discretion, in whole points.
5. Pairs receive the same score; both members must be able to explain any part when asked.